

Lecture 13

Bioinformatics File Formats

The contracts that hold every pipeline together

Computing for Data-Driven Biology · 5BI00A
Master's Programme in Bioinformatics · Umeå University

Session Aims & ILO Links

1 Explain why standardised file formats are essential for bioinformatics pipelines

ILO 4

2 Describe the structure and purpose of FASTA, FASTQ, SAM/BAM, VCF, GFF/GTF, and BED

ILO 4

3 Interpret Phred quality scores and explain their significance for data quality

ILO 4, 9

4 Parse and inspect common bioinformatics files using grep and awk

ILO 7, 9

5 Use samtools for BAM operations: view, sort, index, and flagstat

ILO 7

6 Commit format exercises and documented commands to Git

ILO 12

You have already used FASTA and FASTQ. Today we deepen that knowledge and extend to the full format toolkit.

Which of these formats have you previously encountered?

FASTA

FASTQ

SAM

BAM

VCF

GFF / GTF

BED

None of the above

Also: word cloud — in one word, what IS a bioinformatics file format?

Part 1

Why file formats matter — contracts between tools

The Pipeline as a Series of Format Conversions

A bioinformatics pipeline is not a single program. It is a chain of tools — and each tool must agree on how data is structured. A file format is the contract that makes this possible.

FASTQ**Raw reads**

Quality-trimmed reads from the sequencer

SAM / BAM**Alignments**

Reads mapped to a reference genome

BAM (.bai)**Sorted & indexed**

Ready for variant calling and coverage analysis

VCF**Variants**

SNPs and indels identified vs the reference

BED / GFF / GTF**Features**

Genomic regions, annotations, and gene models

Text vs Binary Formats

Text formats

Human-readable · Inspectable with head, grep, awk ·
Transparent · Debuggable

FASTA · FASTQ · SAM · VCF · GFF · GTF · BED

```
$ $ head -4 reads.fastq
$ $ grep '^#' variants.vcf
$ $ awk '$3=="gene"' annotation.gff
```

Binary formats

Compressed · Not directly readable · Indexed for random
access · Required for large-scale analysis

BAM (binary SAM) · BCF (binary VCF)

```
# Cannot be read directly – always use the right tool
$ $ samtools view alignment.bam | head
$ $ bcftools view variants.bcf | head
```

Good habit: always inspect the header of a binary file before processing it — samtools view -H for BAM, bcftools view -h for BCF.

Part 2

FASTA · FASTQ · Quality scores

FASTA: Sequences Without Quality

```
>NP_000537.3 cellular tumor antigen p53 isoform a [Homo sapiens]  
MEEPQSDPSVEPPLSQETFSDLWKLLPENNVLSPLPSQAMDDLMLSPDDIEQWFTEDP  
GQCTNAHYNYMCNSSCMGQMNRRLPILTIITLEDSSGKLLGRNSFEVRVCACPGRRRTEENLRKKGEVVAPQHLIRVEGSQLAQDDCMFGRNSFEVRVCACPGRRRTEEN  
LRKKGEVVAPQ
```

- ▶ Header line begins with > — everything after the first space is the description (ignored by most tools)
- ▶ Identifier is the first whitespace-delimited token — e.g. 'NP_000537.3'. This is what tools use as the sequence name.
- ▶ Sequence can span multiple lines (usually wrapped at 60 or 80 characters). The line length is formatting, not data.
- ⚠ Lowercase letters = soft-masked repeats (still used but flagged). Uppercase N = hard-masked unknown sequence.
- ⚠ Different databases use different header formats. Parsers that extract the identifier may fail if the format differs.

FASTQ: Sequences With Quality Scores

Each read occupies exactly four lines — always four, never more, never fewer.

```
@SRR8615409.1 1 length=150
```

```
TCCAGACGCAGAGCGGGGCGCAAATCCCGGCAACAGTCCGAGGGAAATAATGGGCCA
```

```
+
```

```
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
```

Line 1: Header — begins with @ (NOT >). Read name and optional description.

Line 2: Sequence — the actual base calls.

Line 3: Separator — always +, optionally repeated header. Often just a bare +.

Line 4: Quality scores — one ASCII character per base.

Parsing FASTQ with shell tools:

```
# Count reads (4 lines per read)
$ wc -l reads.fastq | awk '{print $1/4}'
# Extract sequence lines only
$ awk 'NR%4==2' reads.fastq | head -5
# Extract quality score lines only
$ awk 'NR%4==0' reads.fastq | head -5
# Get basic stats with seqkit
$ seqkit stats reads.fastq
```

Phred Quality Scores: Reading the Confidence

$Q = -10 \times \log_{10}(P)$ where P is the probability that the base call is incorrect

Q score	Error prob.	Accuracy	ASCII char (Phred+33)	Typical use
10	1 in 10	90%	"+" (ASCII 43)	Poor quality — often filtered
20	1 in 100	99%	"5" (ASCII 53)	Low quality — minimum threshold
30	1 in 1,000	99.9%	"?" (ASCII 63)	Good quality — common benchmark
40	1 in 10,000	99.99%	"I" (ASCII 73)	Excellent — high-quality data

⚠ Phred+33 (modern, post-2011): ASCII = $Q + 33$. Phred+64 (old Illumina): ASCII = $Q + 64$.
Mixing encodings is a classic pipeline error — always verify which encoding your data uses.

Q30 is the commonly cited quality threshold — 1 error per 1,000 bases. Modern sequencers typically achieve Q30 for most of a read, with quality declining towards the 3' end.

 Quick Check — Before the Break

Q1. A FASTQ quality character is 'I'. Using Phred+33 encoding, what is the Q score?

A) 9

B) 40 ← ASCII('I')=73; 73-33=40 ✓

C) 33

D) 73

Q2. A SAM alignment has FLAG = 16. What does this tell you?

A) The read is unmapped

B) The read is on the reverse strand ✓ (0x10 = 16)

C) The read is a PCR duplicate

D) The read failed quality filters

Q3. A BED file entry reads: chr1 1000 2000. How many bases does this region cover?

A) 1,001 bases

B) 1,000 bases ✓ (0-based half-open: 2000-1000)

C) 999 bases

D) Cannot tell without knowing the coordinate system



Break — 10 minutes

Back at: _____

After the break: SAM/BAM • VCF • GFF/GTF • BED • hands-on exercise

Part 3

SAM/BAM · VCF · GFF/GTF · BED

SAM / BAM: Alignment Formats

SAM = Sequence Alignment/Map · BAM = Binary Alignment/Map (compressed SAM)

SAM file structure

@HD

File-level metadata (version, sort order)

@SQ

Reference sequence dictionary (chromosome names/lengths)

@RG

Read group (sample, library, sequencing platform)

@PG

Programme chain — which tools produced this file

11 mandatory tab-separated fields per alignment

Key alignment fields

FLAG

Bitwise encoding of alignment properties (paired? mapped? reverse strand? duplicate?)

MAPQ

Mapping quality — probability that the alignment position is wrong

CIGAR

Compact string encoding the alignment: M=match, I=insertion, D=deletion, N=intron (RNA-seq), S=soft clip

POS

1-based leftmost mapping position on the reference

VCF: Variant Call Format

Standard format for recording SNPs, indels, and structural variants identified against a reference genome.

```
## fileformat=VCFv4.3
```

#CHROM	POS	ID	REF	ALT	QUAL	FILTER	INFO	FORMAT	SAMPLE
chr17	7674220	rs28934578	C	T	.	PASS	DP=245;AF=0.45	GT:DP:GQ	0/1:245:99

CHROM/POS

Chromosome and 1-based position of the variant.

REF / ALT

Reference allele and alternate allele(s). Length determines SNP vs indel.

FILTER

PASS if the variant passes all quality filters. Otherwise the filter name(s) that failed.

INFO

Semicolon-delimited key=value pairs — depth (DP), allele frequency (AF), and many others.

FORMAT / SAMPLE

GT=genotype (0=ref, 1=first alt). 0/1 = heterozygous. 1/1 = homozygous alt.

GFF/GTF and BED: Annotation and Intervals

GFF3 / GTF — Genome Annotation

```
chr1  Ensembl  gene  11869  14409  .  +  .  gene_id "ENSG00000223972"; gene_name "DDX11L1";
```

seqname

source

feature

start

end

score

strand

frame

attributes

9 tab-separated columns · Feature types: gene, transcript, exon, CDS, UTR · Attributes: gene_id, gene_name, transcript_id · GFF3 uses key=value; GTF uses key "value" syntax · Both are 1-based coordinates

BED — Genomic Intervals

```
chr1  11869  14409  DDX11L1  0  +
```

⚠ BED uses 0-based, half-open coordinates.

The first base of a chromosome is position 0 (not 1). The end position is exclusive.

This means chr1:1000-2000 in BED covers exactly 1,000 bases (positions 1000 to 1999 in 0-based = 1001 to 2000 in 1-based).

SAM, VCF, GFF, and GTF are 1-based. BED is 0-based. Converting between them without adjusting coordinates is one of the most common off-by-one errors in bioinformatics.

Quick Reference: Formats at a Glance

Format	Type	Primary use	Coordinates	Key tool
FASTA	Text	Sequences (DNA, RNA, protein)	N/A	grep · seqkit
FASTQ	Text	Raw reads with quality scores	N/A	seqkit · FastQC
SAM	Text	Read alignments (human-readable)	1-based	samtools
BAM	Binary	Read alignments (compressed, indexed)	1-based	samtools
VCF	Text	Genomic variants (SNPs, indels)	1-based	bcftools
GFF3/ GTF	Text	Genome annotation features	1-based	grep · awk
BED	Text	Genomic intervals and regions	0-based ⚠	bedtools

⚠ BED is 0-based. Everything else is 1-based. Off-by-one errors from mixing systems are extremely common.

Exercise: Parsing Formats on HPC

Using grep, awk, and samtools to extract meaningful information from each format. All files provided at /proj/nobackup/cddb_course/Bioinformatics_File_Formats/

Part A

Setup

Copy provided files · git init · first commit

Part B

FASTQ inspection

Count reads · inspect 4-line structure · identify quality encoding · check read lengths

Part C

BAM operations

samtools view -H · flagstat · sort · index · view specific region · count mapped reads

Part D

VCF parsing

Count variants · filter PASS · count SNPs vs indels · find variants in TP53 region

Part E

GTF annotation

Count feature types · count genes · extract gene names · find most exon-dense gene

Part F

Document + commit

Complete README with findings · git add · git commit with structured message

Full commands in your handout. The key insight: you can learn a great deal about any file before running any specialist tool.

Exercise: Key Commands

Parts B & C — FASTQ and BAM

```
# Count reads in FASTQ
$ wc -l sample.fastq | awk '{print $1/4}'
# Inspect quality characters (to determine encoding)
$ awk 'NR%4==0' sample.fastq | fold -w1 | sort -u
# BAM: header, stats, sort, index, count mapped
$ module load samtools/1.18
$ samtools view -H sample.bam | grep '^@SQ' | wc -l
$ samtools flagstat sample.bam
$ samtools sort -o sample.sorted.bam sample.bam && samtools index sample.sorted.bam
$ samtools view -c -F 4 sample.sorted.bam # count mapped reads
```

Parts D & E — VCF and GTF

```
# VCF: count variants, PASS only, SNPs vs indels
$ grep -v '^#' sample.vcf | wc -l
$ grep -v '^#' sample.vcf | awk '$7=="PASS"' | wc -l
$ grep -v '^#' sample.vcf | awk 'length($4)==1 && length($5)==1' | wc -l # SNPs
# GTF: feature types, gene count, gene names
$ awk '!/^#{print $3}' sample.gtf | sort | uniq -c | sort -rn
$ awk '!/^#/ && $3=="gene"' sample.gtf | wc -l
$ awk '!/^#/ && $3=="gene"' sample.gtf | grep -oP 'gene_name "\K[^"]+' | head -10
# Commit findings
$ git add . && git commit -m "lecture04: format inspection exercise complete"
```

What You Should Know After This Session



Explain why standardised formats matter and describe a format as a 'contract between tools'



Describe the 4-line FASTQ record structure and convert a quality character to a Q score using Phred+33



Explain what a SAM file contains — header section, mandatory fields, FLAG, and CIGAR (Concise Idiosyncratic Gapped Alignment Report)



Use samtools to view, sort, index, and run flagstat on a BAM file



Describe VCF structure and extract PASS variants, SNPs, and indels using grep and awk



Describe GFF/GTF structure and extract gene counts and names using awk



Explain the BED format and state the 0-based vs 1-based coordinate distinction



Commit format exercise findings to Git with a structured README and meaningful commit message

Next: FAIR Principles, Open Science, and Reproducible Bioinformatics — and why everything you've learned connects to it.